САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2
«Взаимодействие с внешним API»

Выполнил:
Зотеев Максим
Группа K3439

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Привязать сайт из лабораторной работы № 1 к внешнему API средствами fetch, axios или xhr. Реализовать моковое API с помощью JSON-сервера и подключить к нему авторизацию, как в примерах из тем «Иммитация работы с API» и «Иммитация работы с закрытым API».

Вариант остаётся прежним — 4, «Сервис для аренды недвижимости». Все данные, которые в ЛР1 были написаны прямо в HTML, теперь должны приходить с сервера: объявления, карточка объекта, договоры пользователя, переписка и история сделок.

Ход работы

1. Структура проекта

За основу взята вёрстка ЛР1. Запросы выполняются через `fetch`. Скрипты страниц стали ES-модулями (`type="module"`), поэтому общий код вынесен в отдельные файлы и подключается через `import`. Ниже перечислены файлы, которые появились или изменились.

Файл	Назначение
`db.json`	Данные мокового API
`routes.json`	Права доступа к ресурсам
`package.json`	Зависимости и команды запуска
`js/api.js`	Запросы к API и хранение сессии
`js/nav.js`	Шапка: кнопки гостя или пользователя, выход
`js/ui.js`	Форматирование цен и дат, вывод уведомлений
`js/login.js`, `js/register.js`	Вход и регистрация
`js/search.js`	Поиск объявлений
`js/property.js`	Страница объекта, запись на просмотр, сообщение владельцу
`js/profile.js`	Личный кабинет
`js/messages.js`	Диалоги и история сделок

2. Моковое API

Как и в примере с лекции, используются `json-server` и средний слой `json-server-auth`. API запускается командой `npm start` на порту 3000, сайт — командой `npm run site` на порту 8000.

```
  "scripts": {
    "start": "json-server db.json -m ./node_modules/json-server-auth -r
routes.json --port 3000",
    "site": "http-server . -p 8000 -c-1"
  },
```

База состоит из шести коллекций. Вместо хранения одних и тех же данных в разных местах связи заданы полями вида `propertyId`, а JSON-сервер сам подставляет связанный объект по параметру `_expand`. Пароли пользователей в `db.json` хранятся как хеши bcrypt: именно их проверяет `json-server-auth` при входе.

Коллекция	**Что хранит**
`users`	Пользователи
`properties`	Объявления, поле `userId` указывает на владельца
`rentals`	Договоры аренды — это и есть сделки
`conversations`	Диалоги арендатора и владельца по объекту
`messages`	Сообщения диалогов
`viewings`	Записи на просмотр

Права доступа заданы в `routes.json`. Первая цифра кода — права владельца записи, вторая — вошедшего пользователя, третья — гостя. Профиль доступен только самому пользователю (600), объявления видны всем, а менять их может только владелец (644), остальное — только вошедшим (660).

```
{
  "/users*": "/600/users$1",
  "/properties*": "/644/properties$1",
  "/rentals*": "/660/rentals$1",
  "/conversations*": "/660/conversations$1",
  "/messages*": "/660/messages$1",
  "/viewings*": "/660/viewings$1"
}
```

Проверить права можно без браузера. Гость получает ошибку 401, а при попытке создать объявление от имени другого пользователя сервер отвечает 403:

```
GET  /rentals                     → 401 "Missing authorization header"
GET  /users/2     (вошёл id=1)     → 403 "Private resource access..."
POST /properties  {"userId": 2}    → 403 "Private resource creation..."
POST /properties  {"userId": 1}    → 201
```

3. Запросы и сессия

Все запросы проходят через одну функцию `api` из `js/api.js`. Она подставляет токен в заголовок `Authorization`, разбирает ответ и превращает ошибки сервера в понятные пользователю сообщения. Токен живёт час; когда он истекает, сервер отвечает 401, и пользователь отправляется на страницу входа.

```
export async function api(path, { method = 'GET', body } = {}) {
  const headers = {};
  const token = getToken();
  if (body) headers['Content-Type'] = 'application/json';
  if (token) headers.Authorization = `Bearer ${token}`;

  let response;
  try {
    response = await fetch(`${API_URL}${path}`, {
      method,
      headers,
      body: body && JSON.stringify(body),
    });
  } catch {
    throw new Error('Сервер недоступен. Проверьте, что API запущено командой
npm start.');
  }

  const data = await response.json().catch(() => null);

  // Токен json-server-auth живёт час, после этого нужно войти заново
  if (response.status === 401 && token) {
    clearSession();
    redirectToLogin();
  }

  if (!response.ok) {
    const error = new Error(ERROR_MESSAGES[data] ?? 'Не удалось выполнить
запрос. Попробуйте ещё раз.');
    error.status = response.status;
    throw error;
  }

  return data;
}
```

Сессия хранится в браузере. Галочка «Запомнить меня» из ЛР1 теперь работает: с ней токен попадает в `localStorage`, без неё — в `sessionStorage`, и сессия заканчивается вместе с вкладкой.

```
export function saveSession({ accessToken, user }, remember = true) {
  clearSession();
  const store = remember ? localStorage : sessionStorage;
  store.setItem('accessToken', accessToken);
  store.setItem('user', JSON.stringify(user));
}
```

4. Вход и регистрация

Формы отправляются на `/login` и `/register`. В ответе сервер возвращает токен и данные пользователя. Закрытые страницы вызывают `requireAuth`: гость попадает на вход, а после него возвращается туда, куда шёл. Адрес возврата проверяется регулярным выражением, чтобы параметр `next` нельзя было подменить ссылкой на чужой сайт.

```
function nextPage() {
  const next = new URLSearchParams(location.search).get('next');
  return /^[\w-]+\.html(\?[\w=&-]*)?$/.test(next) ? next : 'profile.html';
}
```

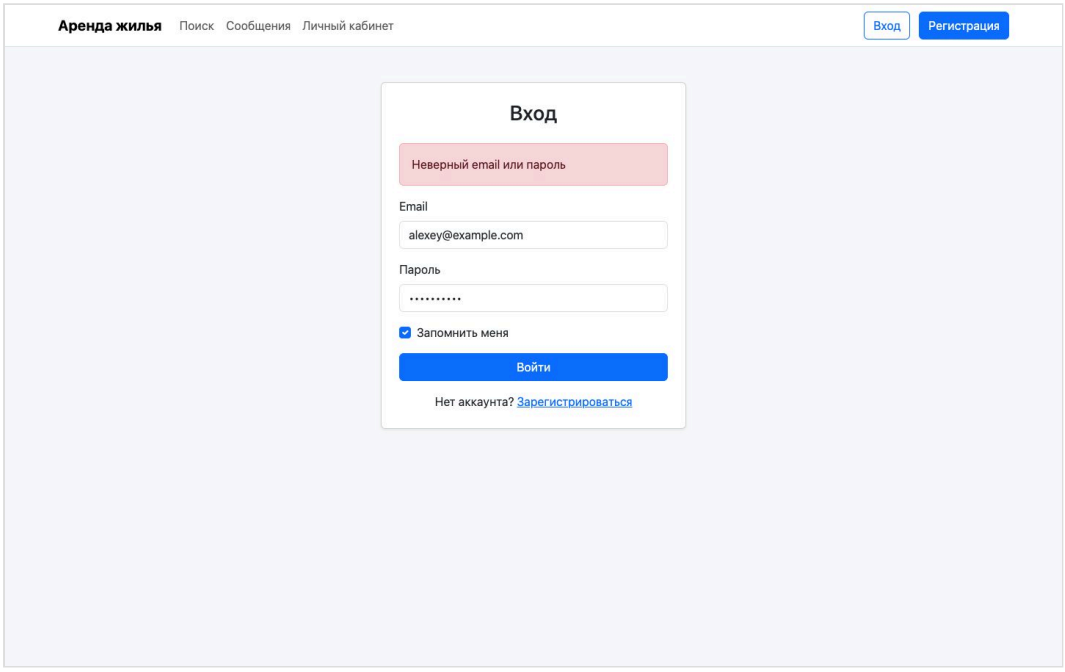


Рисунок 1 — Ошибка входа: сообщение сервера переведено на русский

5. Поиск с фильтрацией

В ЛР1 все карточки лежали в HTML и скрывались скриптом. Теперь фильтрацию выполняет сервер: форма превращается в параметры запроса, а суффиксы `_gte` и `_lte` JSON-сервера задают диапазон цены.

```
function buildQuery() {
  const params = new URLSearchParams();
  if (form.type.value) params.set('type', form.type.value);
  if (form.city.value) params.set('city', form.city.value);
  if (form.priceMin.value) params.set('price_gte', form.priceMin.value);
  if (form.priceMax.value) params.set('price_lte', form.priceMax.value);
  return params;
}
```

Разметка карточки перенесена в тег `<template>`, скрипт клонирует её для каждого объявления. Текст вставляется через `textContent`, поэтому HTML

из данных не выполнится. Также выводятся индикатор загрузки, ошибка при недоступном сервере и сообщение о пустом результате.

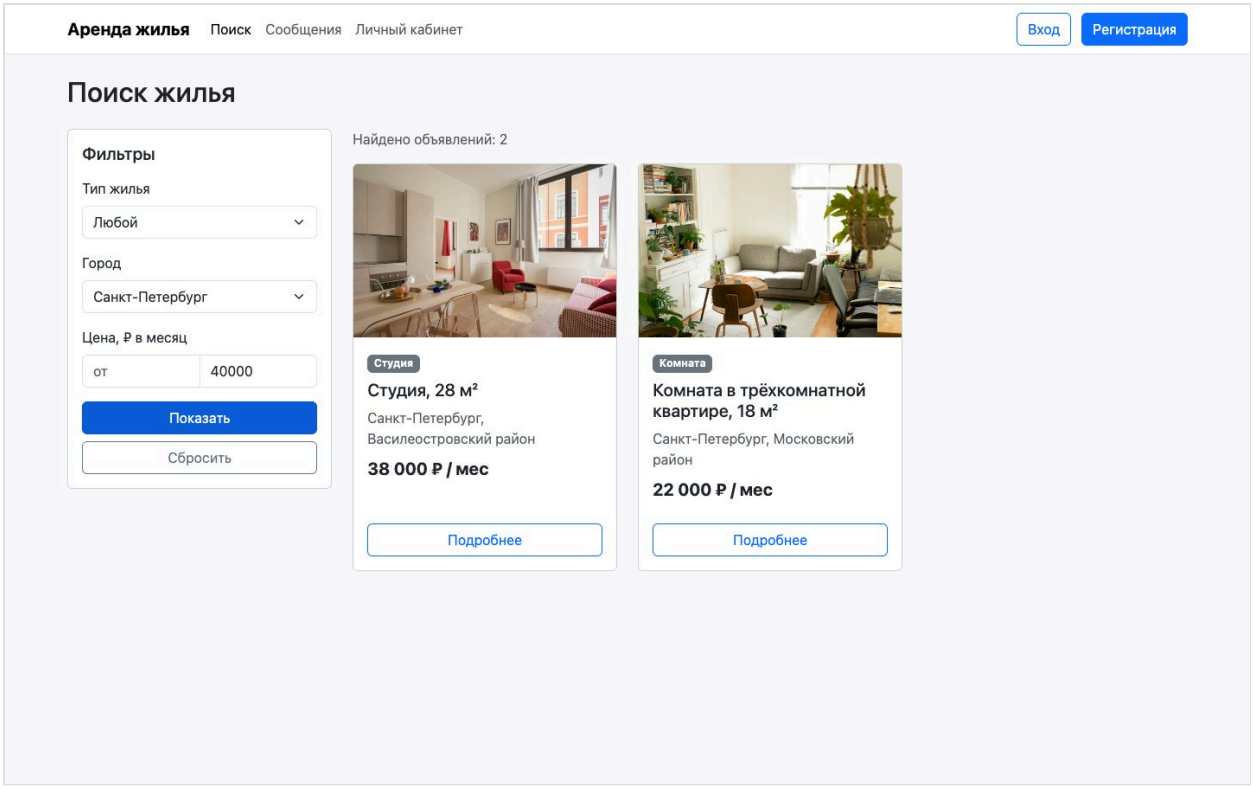


Рисунок 2 — Результат фильтрации: Санкт-Петербург, цена до 40 000 ₽. Запрос: /properties?city=…&price_lte=40000

6. Страница объекта

Страница одна на все объявления: идентификатор передаётся в адресе (`property.html?id=1`). Скрипт запрашивает объект и заполняет заголовок, карусель, условия аренды и характеристики. Гость видит ссылку на вход, владелец — пометку «Это ваше объявление», остальные — кнопки записи и сообщения.

Запись на просмотр сохраняется в `/viewings`, а владельцу уходит сообщение с датой. Оба действия используют одну переписку по объекту: если её нет, она создаётся.

```
async function findConversation(property) {
  const [conversation] = await api(`/conversations?
propertyId=${property.id}&tenantId=${user.id}`);
  if (conversation) return conversation;

  return api('/conversations', {
    method: 'POST',
    body: {
      propertyId: property.id,
      tenantId: user.id,
      tenantName: `${user.firstName} ${user.lastName}`,
      ownerId: property.userId,
      ownerName: property.ownerName,
    },
  });
}
```

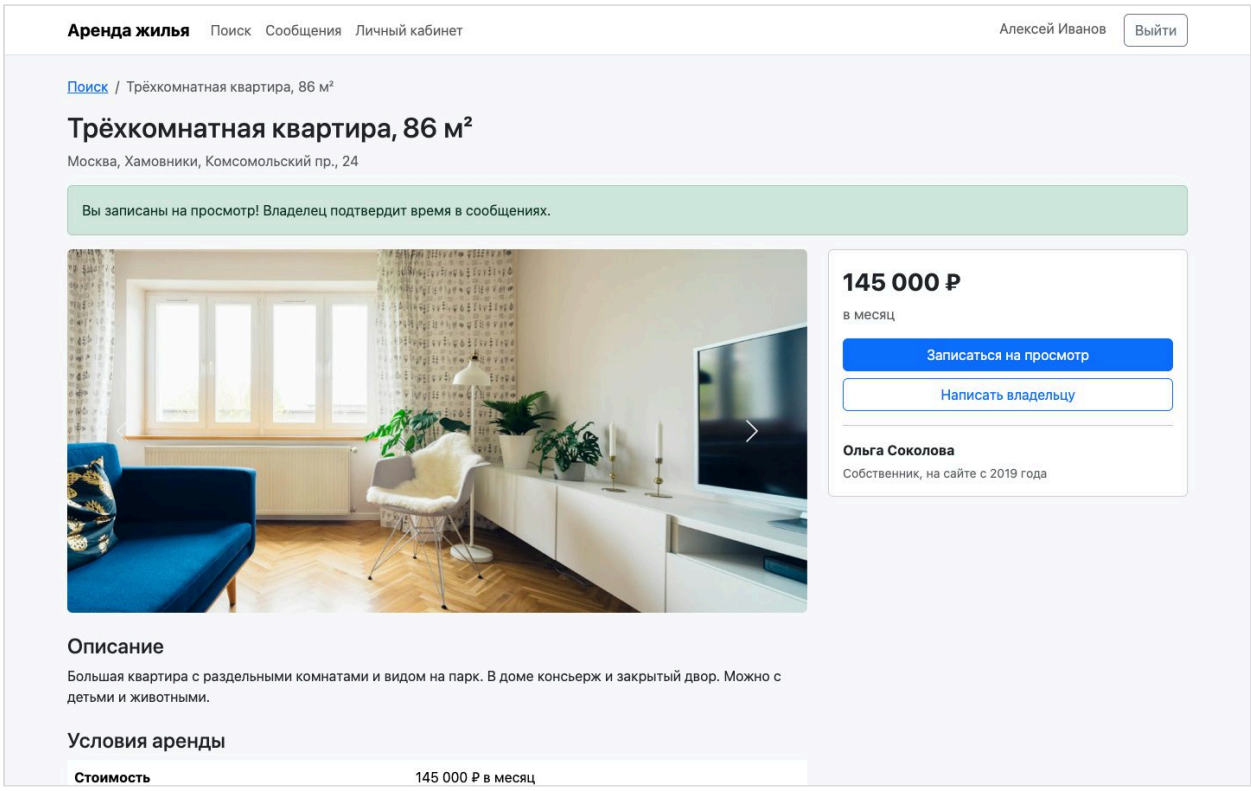


Рисунок 3 — Страница объекта после записи на просмотр

7. Личный кабинет

Страница доступна только после входа. Профиль приходит из `/users/:id`, договоры — из `/rentals` вместе с объектом благодаря `_expand=property`. Вкладка «Настройки» сохраняет изменения запросом PATCH, а модальное окно нового объявления — запросом POST, после чего объявление сразу появляется в поиске. В форму добавлены город и площадь: без них объявление нельзя было бы найти фильтром.

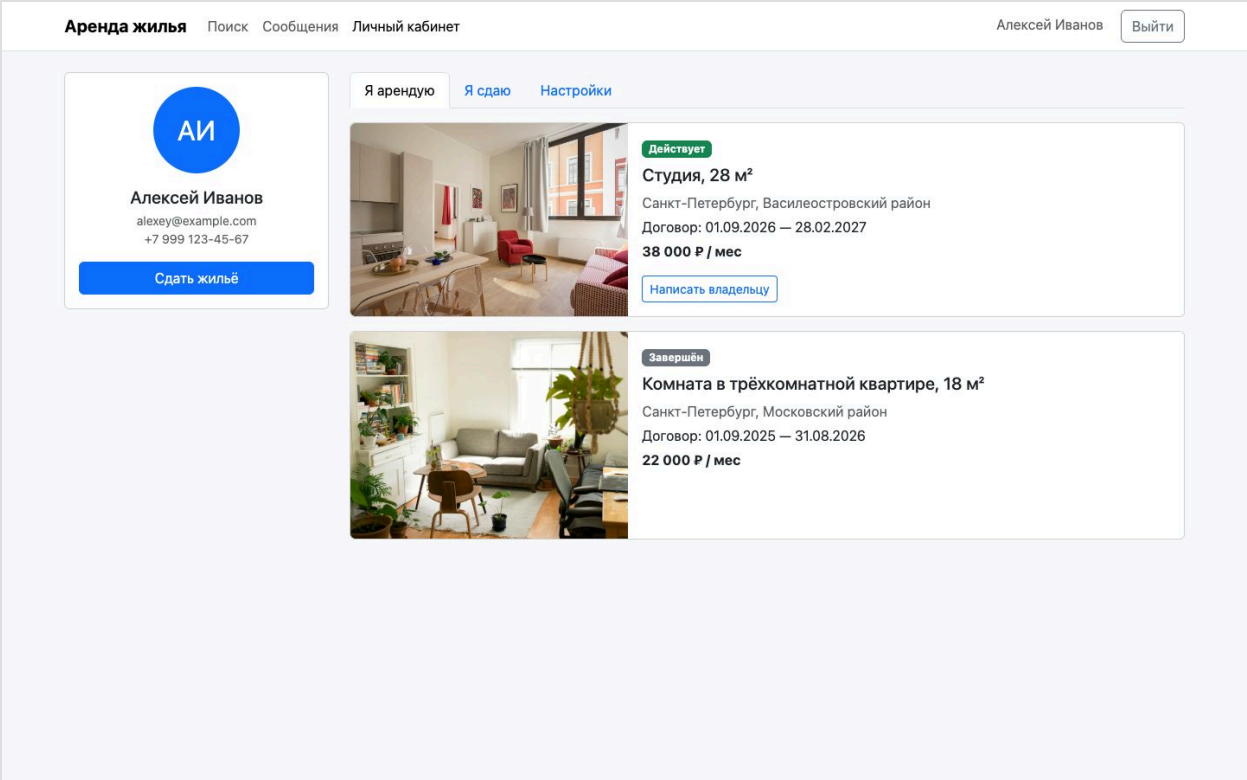


Рисунок 4 — Личный кабинет: вкладка «Я арендую»

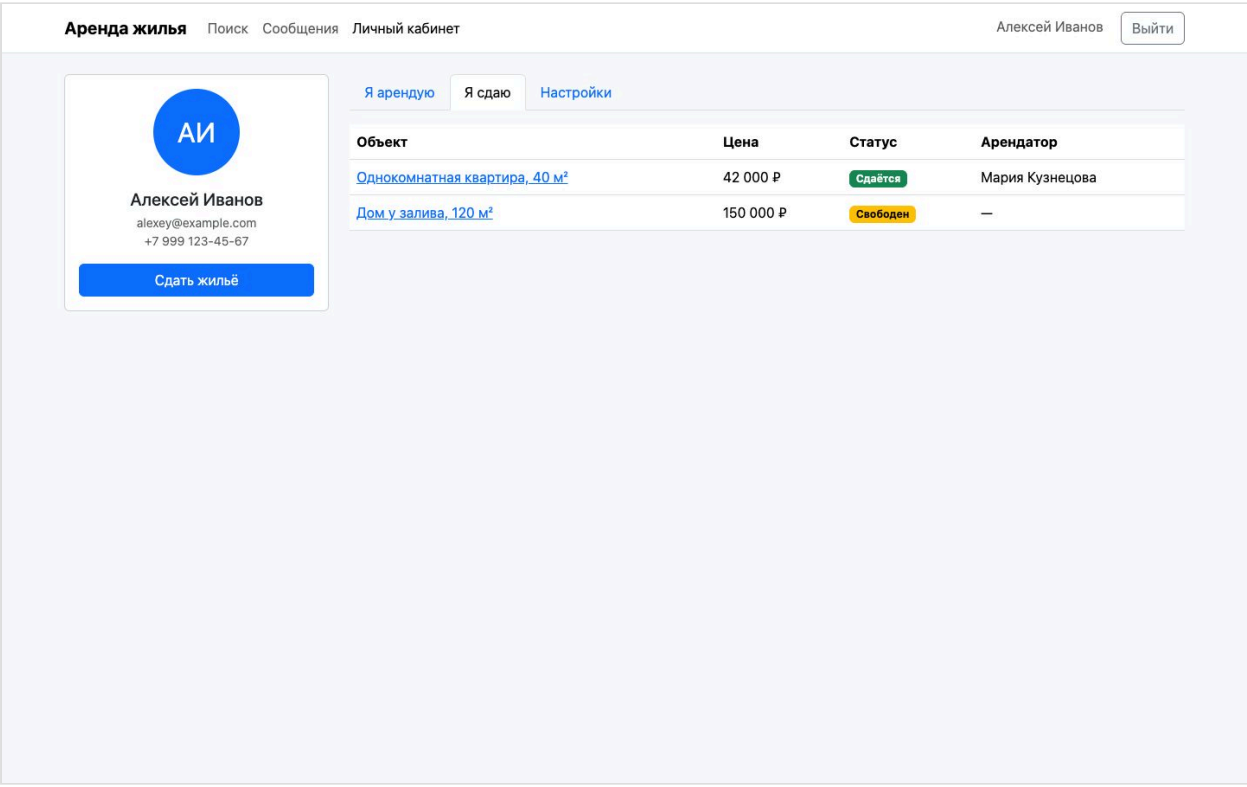


Рисунок 5 — Личный кабинет: вкладка «Я сдаю»

8. Сообщения и история сделок

Пользователь может быть и арендатором, и владельцем, поэтому диалоги и сделки запрашиваются с двух сторон, а результаты объединяются.

Отправленное сообщение сохраняется на сервере и остаётся в чате после перезагрузки страницы.

```
async function loadBothSides(resource, user) {
  const [asTenant, asOwner] = await Promise.all([
    api(`/${resource}?tenantId=${user.id}&_expand=property`),
    api(`/${resource}?ownerId=${user.id}&_expand=property`),
  ]);
  return [...asTenant, ...asOwner];
}
```

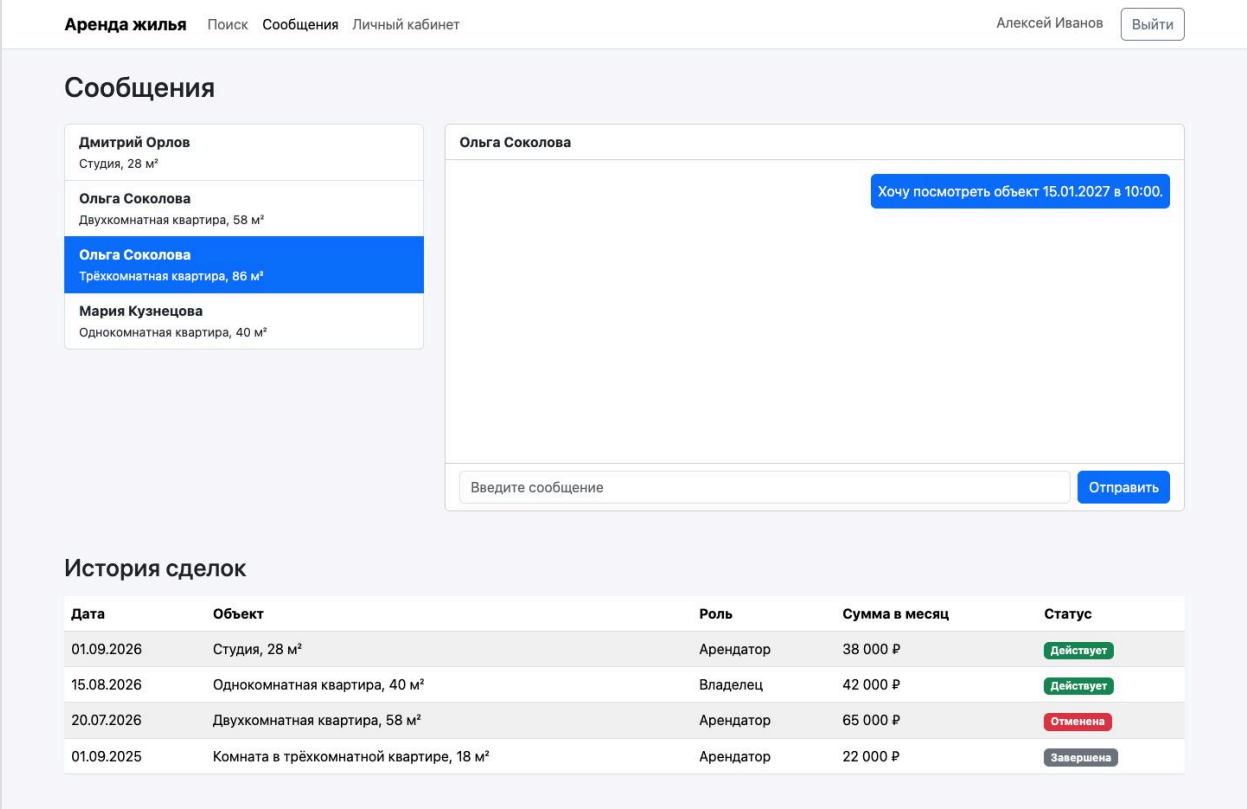


Рисунок 6 — Диалоги, сообщение о записи на просмотр и история сделок

9. Проверка

Сценарии работы сайта были пройдены в браузере Chrome: поиск и сброс фильтров, вход с неверным и верным паролем, возврат на закрытую страницу после входа, регистрация, в том числе с занятым email, изменение настроек, публикация объявления, запись на просмотр, переписка, выход и работа с недействительным токеном. Ошибок в консоли нет.

Вывод

В ходе работы сайт аренды недвижимости был подключён к моковому API на json-server: все данные теперь приходят с сервера и сохраняются в нём. Я разобрался с запросами через fetch, авторизацией по JWT-токену, разграничением прав в json-server-auth, а также научился обрабатывать

состояния загрузки и ошибок и выводить данные через шаблоны `<template>`.